Day 2 Task Workflow — Wednesday, Feb 26, 2026

> **Sprint:** 1 (Foundation) | **Phase:** Database Schema & ORM Models
> **Owner:** Lead Architect | **Status:** ⚪ Ready to Start
> **Goal:** Complete database schema with SQLAlchemy ORM models, Alembic migrations, and Pydantic validation schemas

---

Docker Issue Impact Analysis

Current Situation

- **Issue:** Windows update corruption preventing Docker from running
- **Impact:** Cannot run PostgreSQL + Redis containers locally
- **Severity:** ⚪ Medium — Blocks database verification but NOT code development

Temporary Options (Continue Development)

Option	Description	Pros	Cons
Option A: SQLite Fallback	Use SQLite for local development, PostgreSQL for production	No Docker needed, fast setup	Different DB engine, some PG-specific features unavailable
Option B: Remote PostgreSQL	Use a free cloud PostgreSQL (Supabase, Neon, Railway)	Real PostgreSQL, no local Docker	Requires internet, slight latency
Option C: Code-First Approach	Write all models/migrations now, verify when Docker fixed	No blockers, pure code work	Cannot verify until Docker works
Option D: WSL2 Docker	Run Docker inside WSL2 (if available)	Full Docker experience	Requires WSL2 setup

RECOMMENDED: Option C (Code-First) + Option B (Remote PG for verification)

Rationale:

1. Day 2 tasks are 90% code-writing (models, schemas, migrations)
2. Alembic migrations can be generated without running them
3. SQLAlchemy models can be written and type-checked without DB connection
4. Use remote PostgreSQL (Supabase free tier) for final verification
5. When Docker is fixed, switch back to local PostgreSQL

Implementation Strategy

```
┌──────────────────────────────────────────────────────────────────┐
│ DAY 2 DEVELOPMENT WORKFLOW (Docker-Free)                         │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  Phase 1: Pure Code (No DB Required)                             │
│  ├── Write SQLAlchemy ORM models (all 10 tables)                 │
```

```
│  ├── Write Pydantic request/response schemas                  │
│  ├── Configure Alembic (env.py, script.py.mako)               │
│  └── Generate initial migration (autogenerate)                │
│                                                               │
│  Phase 2: Remote Verification (Supabase/Neon)                 │
│  ├── Create free PostgreSQL instance                          │
│  ├── Update .env with remote connection string                │
│  ├── Run: alembic upgrade head                                │
│  └── Verify tables created successfully                       │
│                                                               │
│  Phase 3: Local Docker (When Fixed)                           │
│  ├── Switch .env back to localhost                            │
│  ├── docker-compose up -d postgres redis                      │
│  ├── alembic upgrade head                                     │
│  └── Full local verification                                  │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

Today's Objective

Complete the database layer so that by end of day:

- All 10 SQLAlchemy ORM models defined with proper types, relationships, indexes
- Alembic configured and initial migration generated
- Pydantic schemas for request/response validation on all entities
- Database connection works with async SQLAlchemy (asyncpg)
- All code passes type checking (mypy/pyright)

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications. No features outside the defined scope.

Requirement	Source Document	Section
10 tables only	GLOBAL_CONTEXT.md	Section 8
Async SQLAlchemy	GLOBAL_CONTEXT.md	Section 2 (Technology Stack)
PostgreSQL 16	GLOBAL_CONTEXT.md	Section 2
Alembic migrations	GLOBAL_CONTEXT.md	Section 2
Pydantic models	GLOBAL_CONTEXT.md	Section 9 (Rule #9)
Type hints everywhere	GLOBAL_CONTEXT.md	Section 9 (Rule #9)

Task Breakdown

TASK 1 — Alembic Configuration ☑

Time Est: 15 min | **Priority:** ⬤ Critical

Configure Alembic for async SQLAlchemy with PostgreSQL.

File	Purpose	Status
`backend/alembic.ini`	Alembic configuration file	☐
`backend/alembic/env.py`	Async migration environment	☐
`backend/alembic/script.py.mako`	Migration template	☐

Key Configuration Points:

- Use `asyncpg` as the driver
- Import `Base` from `app.database`
- Configure `run_migrations_online()` for async
- Set `target_metadata = Base.metadata`

env.py must include:

```
from app.database import Base
from app.models import *  # Import all models for autogenerate

target_metadata = Base.metadata

def run_migrations_online():
    connectable = create_async_engine(
        config.get_main_option("sqlalchemy.url"),
        poolclass=pool.NullPool,
    )
    # ... async migration context
```

Verification: `cd backend && alembic --help` shows Alembic commands

TASK 2 — SQLAlchemy ORM Models 🔨

Time Est: 90 min | **Priority:** 🔴 Critical

Create all 10 database tables as SQLAlchemy ORM models.

2.1 Base Model (`backend/app/models/base.py`)

```
from datetime import datetime
from uuid import uuid4
from sqlalchemy import DateTime, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    """Base class for all models."""
    pass

class TimestampMixin:
    """Adds created_at and updated_at columns."""
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
```

```
        server_default=func.now(),
        nullable=False
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False
    )
```

2.2 User Model (`backend/app/models/user.py`)

Table: `users` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
email	VARCHAR(255)	UNIQUE, NOT NULL	
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
full_name	VARCHAR(255)	NOT NULL	
role	VARCHAR(50)	NOT NULL, default 'viewer'	admin, soc_manager, analyst, viewer
language	VARCHAR(10)	default 'en'	en, am
is_active	BOOLEAN	default true	
last_login	TIMESTAMPTZ	nullable	
created_at	TIMESTAMPTZ	server_default=now()	
updated_at	TIMESTAMPTZ	server_default=now(), onupdate	

Relationships:

- alerts (assigned_to, resolved_by)
- pcap_uploads (uploaded_by)
- llm_conversations (user_id)
- audit_log (user_id)

2.3 NetworkFlow Model (`backend/app/models/flow.py`)

Table: `network_flows` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
timestamp	TIMESTAMPTZ	NOT NULL, indexed DESC	
src_ip	INET	NOT NULL, indexed	
dst_ip	INET	NOT NULL, indexed	
src_port	INTEGER	nullable	

Column	Type	Constraints	Notes
dst_port	INTEGER	nullable	
protocol	SMALLINT	NOT NULL	6=TCP, 17=UDP, 1=ICMP
duration	REAL	nullable	
total_bytes	BIGINT	nullable	
total_packets	INTEGER	nullable	
src_bytes	BIGINT	nullable	
dst_bytes	BIGINT	nullable	
features	JSONB	NOT NULL	Full 40+ feature vector
anomaly_score	REAL	nullable, indexed DESC	0.0-1.0
is_anomaly	BOOLEAN	default false, indexed (WHERE true)	
ml_model	VARCHAR(50)	nullable	
label	VARCHAR(50)	nullable	attack type
source	VARCHAR(20)	default 'live'	live, pcap, agent
created_at	TIMESTAMPTZ	server_default=now()	

Indexes:

- `idx_flows_timestamp` ON timestamp DESC
- `idx_flows_src_ip` ON src_ip
- `idx_flows_dst_ip` ON dst_ip
- `idx_flows_anomaly` ON is_anomaly WHERE is_anomaly = true
- `idx_flows_score` ON anomaly_score DESC

2.4 Alert Model (`backend/app/models/alert.py`)

Table: `alerts` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
alert_id	VARCHAR(20)	UNIQUE, NOT NULL	TM-ALERT-00001
severity	VARCHAR(20)	NOT NULL	critical, high, medium, low, info
title	VARCHAR(500)	NOT NULL	
description	TEXT	nullable	
category	VARCHAR(100)	nullable	ddos, port_scan, c2, dns_tunnel, brute_force
source_ip	INET	nullable	
dest_ip	INET	nullable	

Column	Type	Constraints	Notes
confidence	REAL	nullable	0.0-1.0
status	VARCHAR(20)	default 'open'	open, acknowledged, investigating, resolved, false_positive
assigned_to	UUID	FK → users.id	
flow_ids	UUID[]	nullable	related network_flows
ml_model	VARCHAR(50)	nullable	
ai_narrative	TEXT	nullable	LLM-generated explanation
ai_playbook	TEXT	nullable	LLM-generated response steps
resolved_at	TIMESTAMPTZ	nullable	
resolved_by	UUID	FK → users.id	
resolution_note	TEXT	nullable	
created_at	TIMESTAMPTZ	server_default=now()	
updated_at	TIMESTAMPTZ	server_default=now(), onupdate	

2.5 ThreatIntelIOC Model (`backend/app/models/intel.py`)

Table: `threat_intel_iocs` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
ioc_type	VARCHAR(20)	NOT NULL	ip, domain, url, hash, email
ioc_value	VARCHAR(500)	NOT NULL, indexed	
threat_type	VARCHAR(100)	nullable	malware, phishing, c2, scanner, botnet
severity	VARCHAR(20)	nullable	
source	VARCHAR(100)	NOT NULL	otx, abuseipdb, virustotal, manual
source_ref	VARCHAR(500)	nullable	reference URL
first_seen	TIMESTAMPTZ	nullable	
last_seen	TIMESTAMPTZ	nullable	
confidence	REAL	nullable	
tags	TEXT[]	nullable	
raw_data	JSONB	nullable	
is_active	BOOLEAN	default true	
created_at	TIMESTAMPTZ	server_default=now()	
updated_at	TIMESTAMPTZ	server_default=now(), onupdate	

Unique Constraint: UNIQUE(ioc_type, ioc_value, source) **Indexes:**

- `idx_iocs_value` ON ioc_value
- `idx_iocs_type` ON ioc_type

2.6 MLModel Registry (`backend/app/models/ml_model.py`)

Table: `ml_models` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
name	VARCHAR(100)	NOT NULL	isolation_forest_v1
model_type	VARCHAR(50)	NOT NULL	isolation_forest, random_forest, autoencoder
version	VARCHAR(20)	NOT NULL	
status	VARCHAR(20)	default 'training'	training, active, retired, failed
dataset	VARCHAR(100)	nullable	NSL-KDD, CICIDS2017
metrics	JSONB	nullable	{accuracy, precision, recall, f1, auc}
hyperparams	JSONB	nullable	training configuration
file_path	VARCHAR(500)	nullable	path to serialized model
training_time	REAL	nullable	seconds
inference_time	REAL	nullable	avg ms per prediction
is_active	BOOLEAN	default false	currently used for inference
trained_at	TIMESTAMPTZ	nullable	
created_at	TIMESTAMPTZ	server_default=now()	

2.7 CaptureSession Model (`backend/app/models/capture.py`)

Table: `capture_sessions` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
interface	VARCHAR(50)	nullable	
status	VARCHAR(20)	default 'running'	running, stopped, error
packets_total	BIGINT	default 0	
flows_total	BIGINT	default 0	
anomalies_total	INTEGER	default 0	
started_at	TIMESTAMPTZ	server_default=now()	

Column	Type	Constraints	Notes
stopped_at	TIMESTAMPTZ	nullable	
config	JSONB	nullable	BPF filter, timeout settings

2.8 PCAPUpload Model (`backend/app/models/pcap.py`)

Table: `pcap_uploads` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
filename	VARCHAR(500)	NOT NULL	
file_size	BIGINT	nullable	
file_path	VARCHAR(500)	nullable	
status	VARCHAR(20)	default 'pending'	pending, processing, complete, error
packets_count	INTEGER	nullable	
flows_extracted	INTEGER	nullable	
anomalies_found	INTEGER	nullable	
uploaded_by	UUID	FK → users.id	
processed_at	TIMESTAMPTZ	nullable	
created_at	TIMESTAMPTZ	server_default=now()	

2.9 LLMConversation Model (`backend/app/models/conversation.py`)

Table: `llm_conversations` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
user_id	UUID	FK → users.id	
messages	JSONB	NOT NULL	[{role, content, timestamp}]
context_type	VARCHAR(50)	nullable	alert_analysis, threat_hunt, report, general
context_id	UUID	nullable	related alert/flow/report ID
tokens_used	INTEGER	nullable	
cost_usd	REAL	nullable	
provider	VARCHAR(50)	nullable	deepseek, glm, groq
created_at	TIMESTAMPTZ	server_default=now()	
updated_at	TIMESTAMPTZ	server_default=now(), onupdate	

2.10 SystemConfig Model (`backend/app/models/config.py`)

Table: `system_config` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
key	VARCHAR(100)	PK	
value	JSONB	NOT NULL	
description	TEXT	nullable	
updated_at	TIMESTAMPTZ	server_default=now(), onupdate	

2.11 AuditLog Model (`backend/app/models/audit.py`)

Table: `audit_log` | **Source:** MASTER_DOC_PART2 §4.2

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
user_id	UUID	FK → users.id	
action	VARCHAR(100)	NOT NULL	
entity_type	VARCHAR(50)	nullable	
entity_id	UUID	nullable	
details	JSONB	nullable	
ip_address	INET	nullable	
created_at	TIMESTAMPTZ	server_default=now()	

2.12 Models Index (`backend/app/models/__init__.py`)

```
from app.models.base import Base, TimestampMixin
from app.models.user import User
from app.models.flow import NetworkFlow
from app.models.alert import Alert
from app.models.intel import ThreatIntelIOC
from app.models.ml_model import MLModel
from app.models.capture import CaptureSession
from app.models.pcap import PCAPUpload
from app.models.conversation import LLMConversation
from app.models.config import SystemConfig
from app.models.audit import AuditLog

__all__ = [
    "Base",
    "TimestampMixin",
    "User",
    "NetworkFlow",
    "Alert",
    "ThreatIntelIOC",
    "MLModel",
    "CaptureSession",
    "PCAPUpload",
```

```
    "LLMConversation",
    "SystemConfig",
    "AuditLog",
]
```

Verification: `cd backend && python -c "from app.models import *; print('All models imported successfully')"`

TASK 3 — Pydantic Schemas 🔨

Time Est: 60 min | **Priority:** ⬛ Critical

Create request/response validation schemas for all entities.

3.1 Common Schemas (`backend/app/schemas/common.py`)

```python
from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, Field

class PaginationParams(BaseModel):
    """Query parameters for pagination."""
    page: int = Field(default=1, ge=1, description="Page number")
    limit: int = Field(default=50, ge=1, le=100, description="Items per page")

class PaginatedResponse(BaseModel):
    """Paginated response wrapper."""
    items: list
    total: int
    page: int
    limit: int
    pages: int

class TimestampMixin(BaseModel):
    """Mixin for created_at/updated_at fields."""
    created_at: datetime
    updated_at: datetime | None = None

class ErrorResponse(BaseModel):
    """Standard error response."""
    detail: str
    status_code: int
    timestamp: datetime = Field(default_factory=datetime.utcnow)
```

3.2 Auth Schemas (`backend/app/schemas/auth.py`)

```python
from uuid import UUID
from pydantic import BaseModel, EmailStr, Field

class UserCreate(BaseModel):
    """Request schema for user registration."""
```

```
    email: EmailStr
    password: str = Field(min_length=8, max_length=128)
    full_name: str = Field(min_length=1, max_length=255)

class UserLogin(BaseModel):
    """Request schema for user login."""
    email: EmailStr
    password: str

class UserResponse(BaseModel):
    """Response schema for user data."""
    id: UUID
    email: str
    full_name: str
    role: str
    language: str
    is_active: bool
    last_login: datetime | None = None
    created_at: datetime

class TokenResponse(BaseModel):
    """Response schema for JWT tokens."""
    access_token: str
    refresh_token: str
    token_type: str = "bearer"
    expires_in: int

class TokenRefresh(BaseModel):
    """Request schema for token refresh."""
    refresh_token: str
```

3.3 Flow Schemas (`backend/app/schemas/flow.py`)

```
from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, Field
from typing import Any

class FlowResponse(BaseModel):
    """Response schema for network flow data."""
    id: UUID
    timestamp: datetime
    src_ip: str
    dst_ip: str
    src_port: int | None = None
    dst_port: int | None = None
    protocol: int
    duration: float | None = None
    total_bytes: int | None = None
    total_packets: int | None = None
    src_bytes: int | None = None
    dst_bytes: int | None = None
    features: dict[str, Any]
    anomaly_score: float | None = None
    is_anomaly: bool
```

```
    ml_model: str | None = None
    label: str | None = None
    source: str
    created_at: datetime

class FlowListResponse(BaseModel):
    """Response schema for paginated flow list."""
    items: list[FlowResponse]
    total: int
    page: int
    limit: int

class FlowStatsResponse(BaseModel):
    """Response schema for flow statistics."""
    total_flows: int
    anomaly_count: int
    anomaly_percentage: float
    avg_anomaly_score: float | None = None
    protocol_distribution: dict[str, int]
    top_source_ips: list[dict[str, Any]]
    top_dest_ips: list[dict[str, Any]]
```

3.4 Alert Schemas (`backend/app/schemas/alert.py`)

```
from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, Field

class AlertCreate(BaseModel):
    """Request schema for creating an alert."""
    severity: str = Field(pattern="^(critical|high|medium|low|info)$")
    title: str = Field(min_length=1, max_length=500)
    description: str | None = None
    category: str | None = None
    source_ip: str | None = None
    dest_ip: str | None = None
    confidence: float | None = Field(default=None, ge=0.0, le=1.0)

class AlertUpdate(BaseModel):
    """Request schema for updating an alert."""
    status: str | None = Field(default=None,
pattern="^(open|acknowledged|investigating|resolved|false_positive)$")
    assigned_to: UUID | None = None
    resolution_note: str | None = None

class AlertResponse(BaseModel):
    """Response schema for alert data."""
    id: UUID
    alert_id: str
    severity: str
    title: str
    description: str | None = None
    category: str | None = None
    source_ip: str | None = None
    dest_ip: str | None = None
```

```
    confidence: float | None = None
    status: str
    assigned_to: UUID | None = None
    flow_ids: list[UUID] | None = None
    ml_model: str | None = None
    ai_narrative: str | None = None
    ai_playbook: str | None = None
    resolved_at: datetime | None = None
    resolved_by: UUID | None = None
    resolution_note: str | None = None
    created_at: datetime
    updated_at: datetime | None = None

class AlertListResponse(BaseModel):
    """Response schema for paginated alert list."""
    items: list[AlertResponse]
    total: int
    page: int
    limit: int

class AlertStatsResponse(BaseModel):
    """Response schema for alert statistics."""
    total_alerts: int
    open_count: int
    acknowledged_count: int
    investigating_count: int
    resolved_count: int
    false_positive_count: int
    severity_distribution: dict[str, int]
    category_distribution: dict[str, int]
```

3.5 Intel Schemas (`backend/app/schemas/intel.py`)

```
from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, Field
from typing import Any

class IOCResponse(BaseModel):
    """Response schema for threat intelligence IOC."""
    id: UUID
    ioc_type: str
    ioc_value: str
    threat_type: str | None = None
    severity: str | None = None
    source: str
    source_ref: str | None = None
    first_seen: datetime | None = None
    last_seen: datetime | None = None
    confidence: float | None = None
    tags: list[str] | None = None
    raw_data: dict[str, Any] | None = None
    is_active: bool
    created_at: datetime
    updated_at: datetime | None = None
```

```python
class IOCListResponse(BaseModel):
    """Response schema for paginated IOC list."""
    items: list[IOCResponse]
    total: int
    page: int
    limit: int

class IPReputationResponse(BaseModel):
    """Response schema for IP reputation lookup."""
    ip: str
    is_malicious: bool
    confidence: float
    threat_types: list[str]
    sources: list[str]
    first_seen: datetime | None = None
    last_seen: datetime | None = None
    related_iocs: list[IOCResponse] | None = None
```

3.6 ML Schemas (`backend/app/schemas/ml.py`)

```python
from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, Field
from typing import Any

class MLModelResponse(BaseModel):
    """Response schema for ML model data."""
    id: UUID
    name: str
    model_type: str
    version: str
    status: str
    dataset: str | None = None
    metrics: dict[str, Any] | None = None
    hyperparams: dict[str, Any] | None = None
    file_path: str | None = None
    training_time: float | None = None
    inference_time: float | None = None
    is_active: bool
    trained_at: datetime | None = None
    created_at: datetime

class MLModelListResponse(BaseModel):
    """Response schema for paginated ML model list."""
    items: list[MLModelResponse]
    total: int

class PredictionRequest(BaseModel):
    """Request schema for ML prediction."""
    features: dict[str, Any] = Field(description="40+ feature vector")

class PredictionResponse(BaseModel):
    """Response schema for ML prediction result."""
    flow_id: UUID | None = None
```

```
    anomaly_score: float = Field(ge=0.0, le=1.0)
    is_anomaly: bool
    label: str | None = None
    confidence: float = Field(ge=0.0, le=1.0)
    model_used: str
    ensemble_scores: dict[str, float] | None = None
```

3.7 Capture Schemas (`backend/app/schemas/capture.py`)

```
from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, Field
from typing import Any

class CaptureSessionResponse(BaseModel):
    """Response schema for capture session data."""
    id: UUID
    interface: str | None = None
    status: str
    packets_total: int
    flows_total: int
    anomalies_total: int
    started_at: datetime
    stopped_at: datetime | None = None
    config: dict[str, Any] | None = None

class CaptureStartRequest(BaseModel):
    """Request schema for starting capture."""
    interface: str = Field(description="Network interface to capture on")
    bpf_filter: str | None = Field(default=None, description="BPF filter expression")
    timeout: int | None = Field(default=30, description="Flow timeout in seconds")

class PCAPUploadResponse(BaseModel):
    """Response schema for PCAP upload."""
    id: UUID
    filename: str
    file_size: int | None = None
    file_path: str | None = None
    status: str
    packets_count: int | None = None
    flows_extracted: int | None = None
    anomalies_found: int | None = None
    uploaded_by: UUID | None = None
    processed_at: datetime | None = None
    created_at: datetime
```

3.8 Schemas Index (`backend/app/schemas/__init__.py`)

```
from app.schemas.common import (
    PaginationParams,
    PaginatedResponse,
    TimestampMixin,
```

```
    ErrorResponse,
)
from app.schemas.auth import (
    UserCreate,
    UserLogin,
    UserResponse,
    TokenResponse,
    TokenRefresh,
)
from app.schemas.flow import (
    FlowResponse,
    FlowListResponse,
    FlowStatsResponse,
)
from app.schemas.alert import (
    AlertCreate,
    AlertUpdate,
    AlertResponse,
    AlertListResponse,
    AlertStatsResponse,
)
from app.schemas.intel import (
    IOCResponse,
    IOCListResponse,
    IPReputationResponse,
)
from app.schemas.ml import (
    MLModelResponse,
    MLModelListResponse,
    PredictionRequest,
    PredictionResponse,
)
from app.schemas.capture import (
    CaptureSessionResponse,
    CaptureStartRequest,
    PCAPUploadResponse,
)

__all__ = [
    # Common
    "PaginationParams",
    "PaginatedResponse",
    "TimestampMixin",
    "ErrorResponse",
    # Auth
    "UserCreate",
    "UserLogin",
    "UserResponse",
    "TokenResponse",
    "TokenRefresh",
    # Flows
    "FlowResponse",
    "FlowListResponse",
    "FlowStatsResponse",
    # Alerts
    "AlertCreate",
    "AlertUpdate",
```

```
    "AlertResponse",
    "AlertListResponse",
    "AlertStatsResponse",
    # Intel
    "IOCResponse",
    "IOCListResponse",
    "IPReputationResponse",
    # ML
    "MLModelResponse",
    "MLModelListResponse",
    "PredictionRequest",
    "PredictionResponse",
    # Capture
    "CaptureSessionResponse",
    "CaptureStartRequest",
    "PCAPUploadResponse",
]
```

Verification: `cd backend && python -c "from app.schemas import *; print('All schemas imported successfully')"`

TASK 4 — Initial Alembic Migration 🔨

Time Est: 15 min | **Priority:** 🔴 Critical

Generate the initial database migration from all models.

Step	Command	Expected Output
4.1	`cd backend && alembic revision --autogenerate -m "initial_schema"`	Migration file created
4.2	Review migration file	All 10 tables present
4.3	`alembic upgrade head` (when DB available)	Tables created

Migration file location: `backend/alembic/versions/XXXXXX_initial_schema.py`

Verification checklist:

- ☐ All 10 tables in migration
- ☐ All indexes present
- ☐ All foreign keys correct
- ☐ All constraints applied

TASK 5 — Database Connection Test 🔨

Time Est: 15 min | **Priority:** 🟡 Medium

Test database connection with async SQLAlchemy.

Options (based on Docker status):

Option	Connection String	When to Use

Option	Connection String	When to Use
Local Docker	`postgresql+asyncpg://threatmatrix:threatmatrix@localhost:5432/threatmatrix`	Docker working
Remote (Supabase)	`postgresql+asyncpg://user:pass@host:5432/db`	Docker broken
SQLite (fallback)	`sqlite+aiosqlite:///./threatmatrix.db`	Last resort

Test script (`backend/test_db.py`):

```
import asyncio
from app.database import engine
from app.models import Base

async def test_connection():
    async with engine.begin() as conn:
        # Create all tables
        await conn.run_sync(Base.metadata.create_all)
        print("✅ Database connection successful")
        print("✅ All tables created")

if __name__ == "__main__":
    asyncio.run(test_connection())
```

Verification: `cd backend && python test_db.py`

TASK 6 — Type Checking & Linting 🔨

Time Est: 10 min | **Priority:** 🟢 Medium

Ensure all code passes type checking and linting.

Check	Command	Expected
Type check	`cd backend && mypy app/models app/schemas`	No errors
Lint	`cd backend && ruff check app/models app/schemas`	No errors
Format	`cd backend && ruff format app/models app/schemas`	Code formatted

Files Created Today

```
threatmatrix-ai/
├── backend/
│   ├── alembic/
│   │   ├── env.py                    🔨 Task 1
│   │   ├── script.py.mako            🔨 Task 1
│   │   └── versions/
```

```
│   │       └── XXXXXX_initial_schema.py  🔨 Task 4
│   ├── alembic.ini                  🔨 Task 1
│   ├── app/
│   │   ├── models/
│   │   │   ├── __init__.py          🔨 Task 2
│   │   │   ├── base.py              🔨 Task 2
│   │   │   ├── user.py              🔨 Task 2
│   │   │   ├── flow.py              🔨 Task 2
│   │   │   ├── alert.py             🔨 Task 2
│   │   │   ├── intel.py             🔨 Task 2
│   │   │   ├── ml_model.py          🔨 Task 2
│   │   │   ├── capture.py           🔨 Task 2
│   │   │   ├── pcap.py              🔨 Task 2
│   │   │   ├── conversation.py      🔨 Task 2
│   │   │   ├── config.py            🔨 Task 2
│   │   │   └── audit.py             🔨 Task 2
│   │   └── schemas/
│   │       ├── __init__.py          🔨 Task 3
│   │       ├── common.py            🔨 Task 3
│   │       ├── auth.py              🔨 Task 3
│   │       ├── flow.py              🔨 Task 3
│   │       ├── alert.py             🔨 Task 3
│   │       ├── intel.py             🔨 Task 3
│   │       ├── ml.py                🔨 Task 3
│   │       └── capture.py           🔨 Task 3
│   └── test_db.py                   🔨 Task 5
```

Decisions Made Today

Decision	Choice	Reason
Docker workaround	Code-first + remote PG verification	Continue development without local Docker
Model style	SQLAlchemy 2.0 Mapped columns	Modern, type-safe approach
Schema style	Pydantic v2 BaseModel	FastAPI native, auto-validation
Migration strategy	Autogenerate from models	Single source of truth

---

Blockers

Blocker	Severity	Mitigation	Status
Docker not running (Windows update corruption)	🟤 Medium	Use remote PostgreSQL or code-first approach	Mitigated

---

Tomorrow's Preview (Day 3)

- FastAPI auth system: register, login, refresh token, logout
- JWT middleware with access/refresh token flow
- RBAC decorators: `require_role(["admin", "analyst"])`
- Auth dependencies for protected endpoints
- Password hashing with bcrypt

Scope Adherence Verification

> **Every item below MUST be verified before marking task complete.**

Requirement	Source	Verification
10 tables exactly	GLOBAL_CONTEXT.md §8	Count models in `app/models/`
Async SQLAlchemy	GLOBAL_CONTEXT.md §2	Check `asyncpg` in requirements.txt
PostgreSQL 16	GLOBAL_CONTEXT.md §2	Check docker-compose.yml
Alembic migrations	GLOBAL_CONTEXT.md §2	Check alembic/ directory exists
Pydantic models	GLOBAL_CONTEXT.md §9	Check `app/schemas/` directory
Type hints everywhere	GLOBAL_CONTEXT.md §9	Run mypy on models + schemas
UUID primary keys	MASTER_DOC_PART2 §4.2	Check all models use UUID
JSONB for flexible data	MASTER_DOC_PART2 §4.2	Check features, metrics, config columns
Proper indexes	MASTER_DOC_PART2 §4.2	Check index definitions in models
Foreign keys	MASTER_DOC_PART2 §4.2	Check relationship definitions

---

Task workflow for Day 2 — ThreatMatrix AI Sprint 1